



Clases Abstractas e Interfaces en Java

Dos herramientas fundamentales para el diseño orientado a objetos

Clase Abstracta: Concepto Fundamental

¿Qué es?

Una clase que sirve como plantilla pero que no puede instanciarse directamente. Actúa como base para otras clases.

No instanciable

No puedes crear objetos directamente de ella con `new`

Métodos mixtos

Combina métodos abstractos (sin implementar) y concretos (ya implementados)

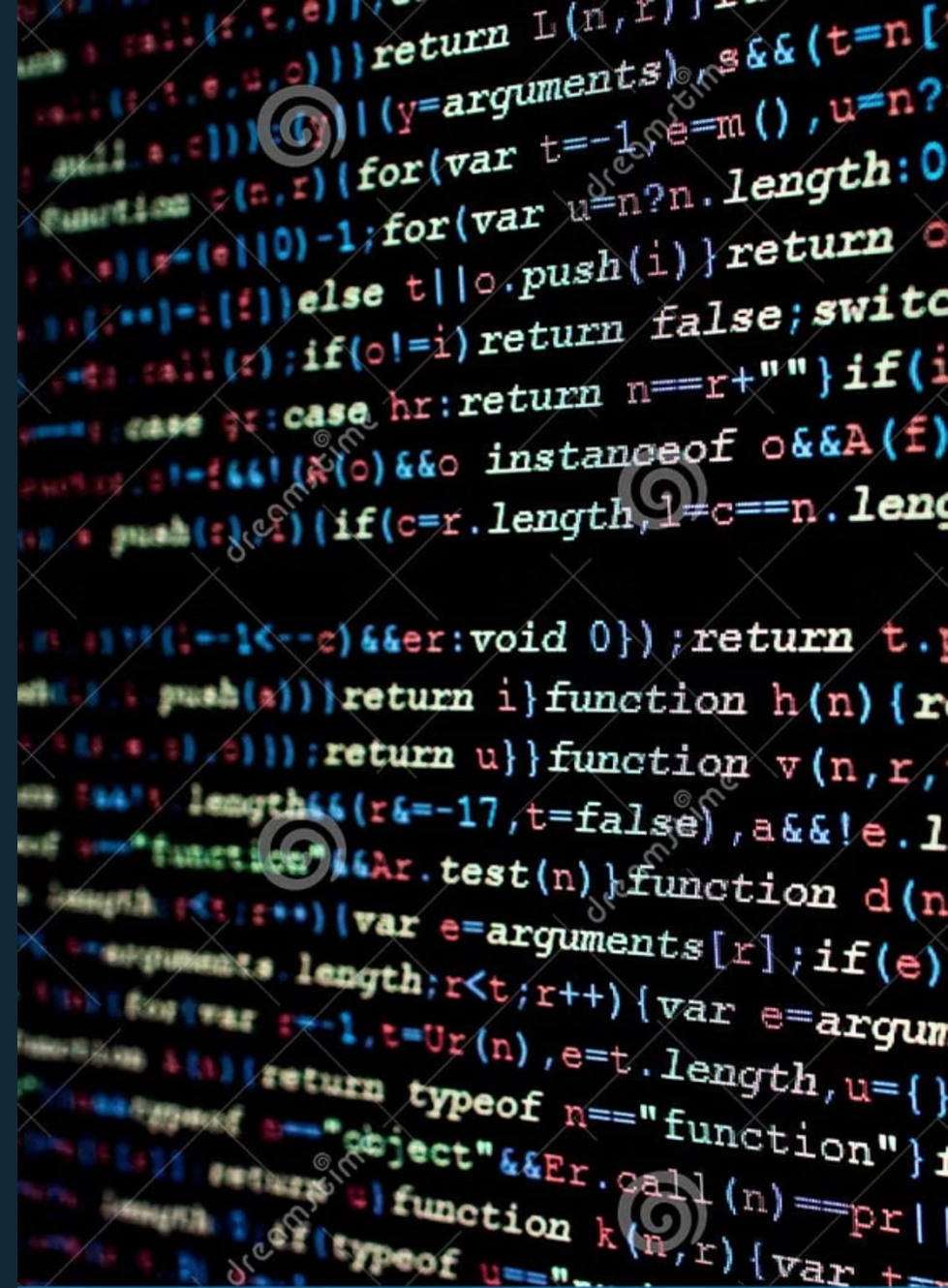
Las clases abstractas son perfectas cuando tienes una jerarquía de clases que comparten código común, pero necesitas forzar que las subclasses implementen ciertos métodos específicos.

Sintaxis de una Clase Abstracta

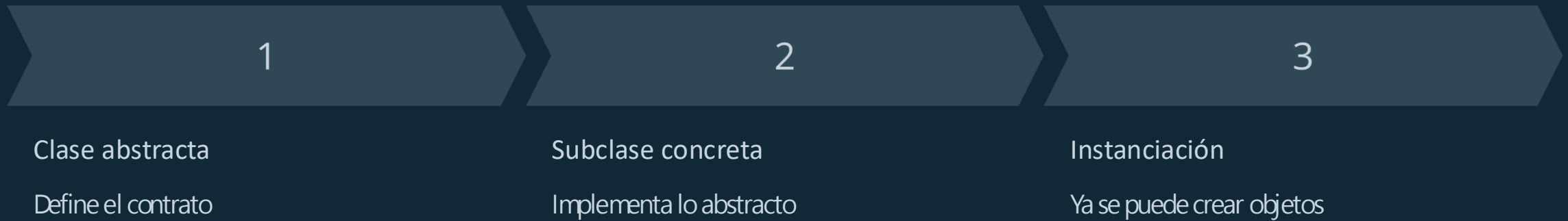
Declaramos una clase abstracta con la palabra clave `abstract`. Puede contener métodos abstractos (sin cuerpo) que las subclases deben implementar, y concretos que todas las subclases heredan.

```
public abstract class Figura {  
    public abstract double area();  
  
    public void trasladar() {  
        System.out.println("Moviendo figura");  
    }  
}
```

Observa cómo `area()` no tiene implementación (es abstracto), mientras que `trasladar()` sí la tiene. Las subclases deben implementar `area()`, pero heredan `trasladar()` automáticamente.



Creando una Subclase Concreta



Una subclase concreta debe implementar todos los métodos abstractos de su superclase. Solo entonces podrá instanciarse:

```
public class Circulo extends Figura {  
    private double radio;  
    @Override  
    public double area() {  
        return Math.PI * radio * radio;  
    }  
}
```

La anotación `@Override` indica explícitamente que estamos implementando un método de la superclase, ayudando a prevenir errores.

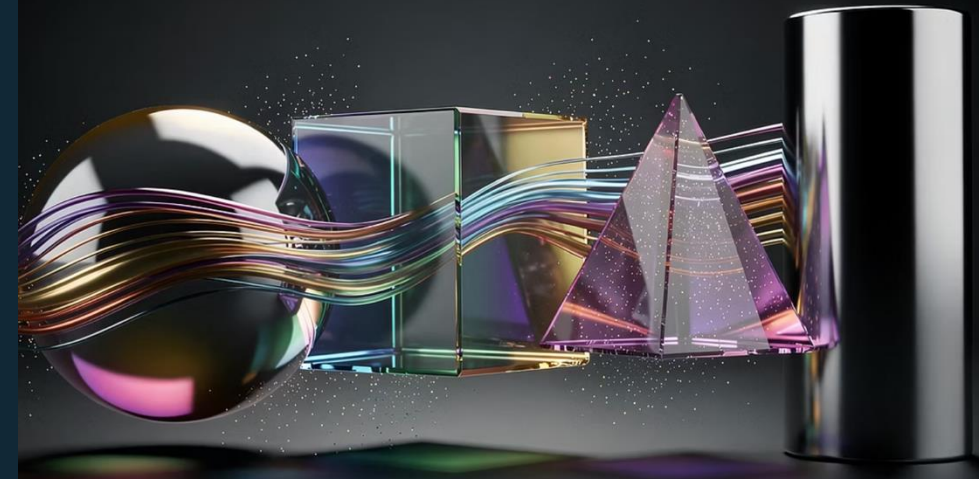
Polimorfismo con Clases Abstractas Abstractas

Aunque no podemos instanciar una clase abstracta directamente, sí podemos usarla como tipo de referencia. Esto es la esencia del polimorfismo:

```
Figura f = new Circulo();
```

Aquí, `f` es del tipo `Figura` (abstracta), pero apunta a un objeto `Circulo` (concreto). (concreto). Esto nos permite escribir código más flexible que trabaja con la lugar de con implementaciones específicas.

El beneficio principal: puedes cambiar la implementación concreta sin cambiar el código que usa la referencia abstracta.



Interfaz: El Concepto de Contrato



Define un contrato

Especifica qué métodos debe tener una clase



Sin implementación

No dice cómo hacerlo, solo qué hacer



Responde "qué"

No "cómo", solo la capacidad que debe existir

Una interfaz es como un contrato que una clase firma. Al implementar una interfaz, la clase promete proporcionar la funcionalidad especificada. Es una forma pura de definir capacidades sin imponer una estructura de implementación.

Definición de una Interfaz

Las interfaces se declaran con la palabra clave `interface`. Por defecto, todos sus métodos son públicos y abstractos:

```
public interface FiguraCerrada {  
    double area();  
    double perimetro();  
}
```

Esta interfaz define un contrato: cualquier clase que implemente `FiguraCerrada` debe proporcionar implementaciones para calcular el área y el perímetro. No importa cómo lo haga, solo que lo haga.

Nota: no necesitas escribir `public abstract` antes de los métodos, es implícito en las interfaces.

REM
JM



Implementación de una Interfaz

Una clase implementa una interfaz usando la palabra clave `implements`. Debe proporcionar código para todos los métodos declarados en la interfaz:

```
public class Circulo implements FiguraCerrada {  
    private double radio;  
  
    public double area() {  
        return Math.PI * radio * radio;  
    }  
    public double perimetro() {  
        return 2 * Math.PI * radio;  
    }  
    // Otros métodos ...  
}
```

Aquí `Circulo` cumple el contrato `FiguraCerrada`. Puede tener sus propios atributos y métodos adicionales, pero como mínimo debe implementar `area()` y `perimetro()`.

Interfaces como Tipo: Desacoplamiento

Polimorfismo flexible

```
FiguraCerrada figura = new Circulo();
```

La variable `figura` es del tipo interfaz, no de la clase concreta.

Desacoplamiento

El código no depende de implementaciones concretas

Flexibilidad

Puedes cambiar la implementación sin cambiar el código cliente

Este enfoque hace tu código más mantenible y testeable. Por ejemplo, un método puede aceptar `FiguraCerrada` como parámetro y funcionará con cualquier clase que implemente esa interfaz: `Circulo`, `Rectangulo`, `Triangulo`, etc.

Clase Abstracta vs Interfaz

Ambas son herramientas de abstracción, pero con propósitos y capacidades diferentes. Entender cuándo usar cada una es clave para un buen diseño:

Característica	Clase Abstracta	Interfaz
Estado (atributos)	✅ Puede tener	❌ No tiene
Propósito	Relación es-un (compartir código)	Qué se debe hacer (definir contrato)
Herencia	Simple (una sola)	Múltiple

Regla práctica: usa clase abstracta cuando necesites compartir código entre clases relacionadas; usa interfaz cuando necesites definir un contrato que clases no relacionadas puedan cumplir.

¿Cuándo Usar una Interfaz?



Contrato común

Cuando múltiples clases necesitan garantizar que implementan las mismas capacidades



Clases no relacionadas

Cuando clases sin relación jerárquica comparten comportamiento



Diseño con Interfaces: Dominio Salud

Veamos un ejemplo práctico del poder de las interfaces en un contexto real. Imagina un sistema de monitorización de salud:

Diferentes mediciones médicas (temperatura corporal, frecuencia cardíaca, presión arterial) no están relacionadas entre sí, pero todas comparten la capacidad de ser medidas. La interfaz **Medible** captura esta característica común sin forzar una jerarquía artificial.

Interfaz Medible

Núcleo que unifica lecturas



Temperatura

Implementación de sensor térmico



Pulsaciones

Implementación de sensor de pulso

Ejemplo Completo: Sistema de Mediciones

Implementación práctica del diseño anterior:

```
public interface Medible {  
    double getValor();  
}  
public class Temperatura implements Medible {  
    private double valor;  
    //...  
    public double getValor() {  
        return valor;  
    }  
}
```

Ahora podemos crear una lista `List<Medible>` que contenga tanto temperaturas como pulsaciones, y procesarlas de forma uniforme. Un método que trabaje con `Medible` funcionará con cualquier tipo de medición, actual o futura.

Este patrón es extensible: añadir nuevos tipos de mediciones (glucosa, oxígeno en sangre) no requiere cambiar código existente.

Diseño con Interfaces: Un Juego Interactivo

Las interfaces son fundamentales en el desarrollo de videojuegos para definir comportamientos comunes sin preocuparse por la implementación específica. Piensa implementación específica. Piensa en ellas como un "contrato" que cualquier objeto puede firmar.

1. Definición de la Interfaz

Creamos una interfaz que establece qué acción se puede realizar:

```
public interface Interactable {  
    void interactuar(); // Solo la firma del método  
}
```

2. Implementación en Diferentes Clases

Ahora, diferentes objetos implementan esta interfaz, definiendo su propia forma de "interactuar":

```
public class Cofre implements Interactable {  
    @Override  
    public void interactuar() {  
        System.out.println("¡Has abierto el cofre y encontrado 100 monedas de oro!");  
    }  
}  
  
public class Aldeano implements Interactable {  
    @Override  
    public void interactuar() {  
        System.out.println("Aldeano: '¡Cuidado con los monstruos en el bosque!'");  
    }  
}
```


Diseño con Interfaces: Un Juego Interactivo

Esta "magia" permite al motor del juego tratar todos los objetos de forma genérica.

Uso en el Código del Juego

```
// El juego detecta un objeto cerca del jugador
Interactable objetoCercano = detectarObjetoInteractableCercaDelJugador();

// No importa qué sea, simplemente invoca su método interactuar()
objetoCercano.interactuar();
```

El juego no necesita saber si interactúa con un cofre, un aldeano o una palanca; solo sabe que es algo **Interactable**.

Flexibilidad

Añade nuevos objetos con facilidad; el código del jugador seguirá funcionando.

Desacoplamiento

El jugador no necesita conocer los detalles internos de cada objeto.

Organización

Impone convenciones claras para acciones similares en todo el equipo de desarrollo.

Combinando Clases Abstractas e Interfaces

No son excluyentes

Una clase puede extender una clase abstracta y implementar interfaces

Se complementan

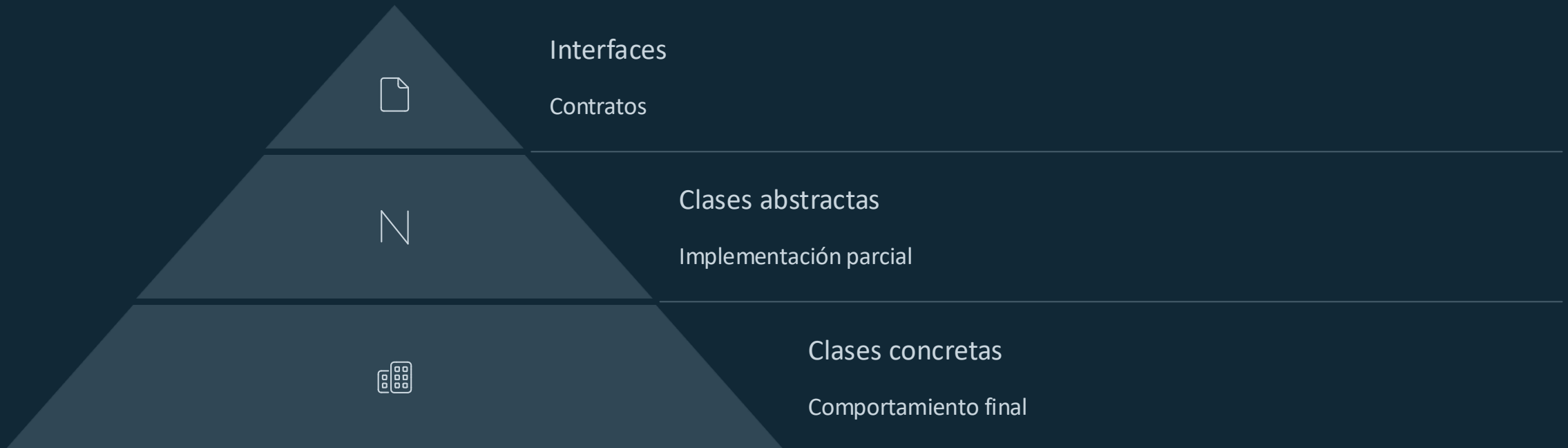
Herencia para compartir Código (es-un), interfaces para garantizar contratos

Ejemplo: `public class Rectangulo extends Figura implements
FiguraCerrada, Dibujable`

Esta sintaxis es perfectamente válida y muy poderosa. La clase abstracta proporciona funcionalidad común, mientras que las interfaces definen capacidades adicionales que la clase debe cumplir.



Principios para un Diseño Limpio



La herencia modela especialización. Las interfaces modelan contratos. El diseño sigue siendo lo primero.

Recuerda: el buen diseño orientado a objetos no se trata de usar todas las herramientas disponibles, sino de elegir la herramienta correcta para cada correcta para cada situación. Piensa primero en el problema, luego en la solución técnica.

Dispositivos

Cada dispositivo debe ser capaz de realizar tres acciones siguiendo su propia naturaleza:

2. **Actualización de Medición:** El dispositivo debe obtener un nuevo valor representativo de la constante vital que supervisa. En nuestro caso, se va a simular este valor. En esta simulación los valores deben fluctuar de forma aleatoria dentro de rangos médicos realistas ([investiga la clase Random](#))
3. **Evaluación de Estado Crítico:** El dispositivo debe analizar su medición actual y determinar si existe una situación de riesgo. Cada tipo de constante (oxígeno, temperatura o pulso) tiene sus propios criterios médicos para decidir qué es "peligroso".
4. **Notificación de Emergencia:** El dispositivo debe proporcionar un mensaje de alerta específico que describa la anomalía detectada y el **identificador (Cama)** que la ha generado.

```
public interface DispositivoMonitor {  
    void actualizarMedicion();  
    boolean esCritico();  
    String notificacionEmergencia();  
}
```

Interfaces como abstracción funcional

Los interfaces proporcionan soluciones elegantes a problemas donde se desea aplicar diferentes funcionalidades a unos mismos datos (**abstracción funcional**).

Para comprenderlo mejor, veamos un ejemplo solucionado sin interfaces y después con interfaces aplicando abstracción funcional.

Disponemos de dos clases

- Clase **Persona** con información de una persona (nombre y edad)
- Clase **Amigos** con información de muchas personas (un conjunto de personas)

Queremos operar con la clase Amigos

- Que son menores de una edad dada
- Que son mayores de una edad dada
- Cuyo nombre empieza por una cadena dada

Nota: El cuerpo de estos métodos será muy parecido, solo cambiarán la manera de seleccionar las personas.

Solución sin interfaces

Lo natural es crear un método en la clase **Amigos** que resuelva cada una de las funcionalidades requeridas:

menoresQue

Para las personas menores de una edad dada

```
Set<Persona> menoresQue(int n)
```

mayoresQue

Para las personas mayores de una edad dada

```
Set<Persona> mayoresQue(int n)
```

empiezaCon

Para las personas cuyo nombre empieza por una cadena dada

```
Set<Persona> empiezaCon(String s)
```

El cuerpo de estos métodos **será muy parecido**, sólo cambiarán la manera de **seleccionar** las personas.

¿Cómo añadir una nueva funcionalidad?

Por ejemplo, para recopilar las personas cuyo nombre **contiene** un determinado texto. Tendremos que modificar la clase **Amigos** y añadir un nuevo método:

```
public List<Persona> contiene(String s) {  
    List<Persona> nueva = new ArrayList<>();  
    for (Persona p : personas) {  
        if (p.getNombre().contains(s)) {  
            nueva.add(p);  
        }  
    }  
    return nueva;  
}
```

La cohesión de la clase disminuye si
quieres más formas de filtrar

Cada nuevo filtro, implica modificar la
clase Amigos

Problema: Cada nueva funcionalidad obliga a modificar y recompilar **Amigos** — esto viola el principio abierto/cerrado.

Solución con interfaces

abstracción funcional

1. Interfaz Seleccionador

Define si una persona se debe seleccionar.

Método:

```
boolean selecciona(Persona p)
```

2. Clases que implementan

Cada funcionalidad se define en una clase que implementa **Seleccionador**, definiendo **selecciona** a su manera.

Clases:

MenoresQue

MayoresQue

EmpiezaCon

3. Clase contenedora

Dispone de un método que toma un **Seleccionador** como argumento y realiza la selección.

En la clase **Amigos**:

Set<Persona>

```
filtra(Seleccionador selec)
```

Ventaja clave: Añadir nuevas funcionalidades no requiere modificar la clase **Amigos**, solo crear nuevas clases que implementen **Seleccionador**.

Interfaz Predicado y clase Amigos

```
public interface Seleccionador {  
    boolean selecciona(Persona p);  
}
```

abstraccionfuncional

abstraccionfuncional/Seleccionador.java

```
public class Amigos {  
    private List<Persona> personas;  
    ...  
    public List<Persona> filtra(Seleccionador selector) {  
        Set<Persona> nueva = new HashSet<Persona>();  
        for (Persona p : personas) {  
            if (selector.selecciona(p)) {  
                nueva.add(p);  
            }  
        }  
        return nueva;  
    }  
}
```

abstraccionfuncional/AmigosConInterfaz.java

¿Cómo añadir la nueva funcionalidad ahora?

Añadimos otra clase que implemente la interfaz **Predicado**:

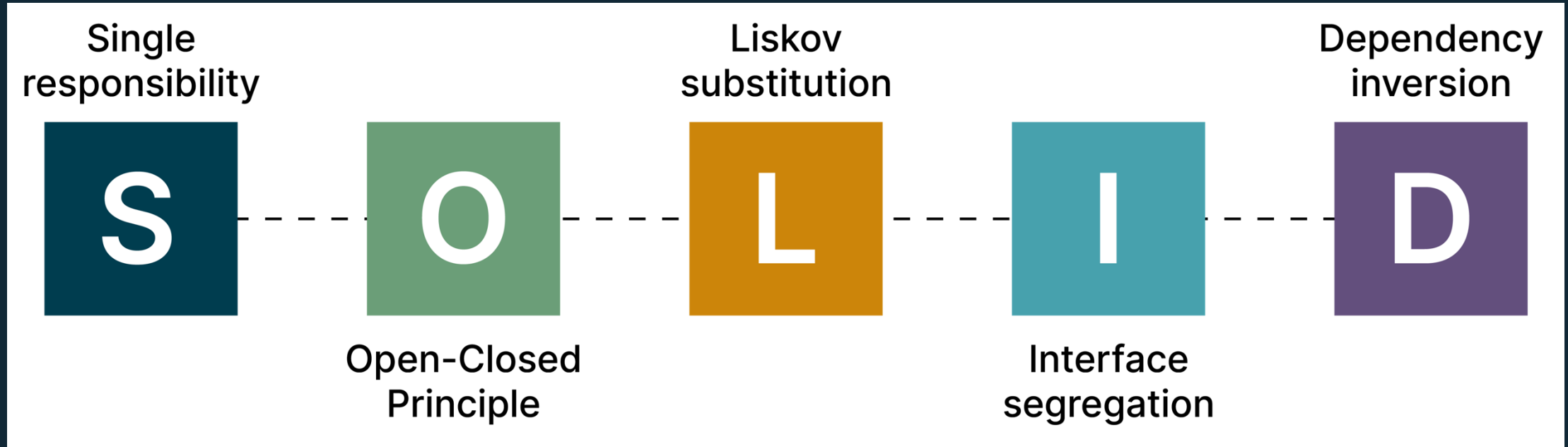
```
public class Contiene implements Seleccionador {  
    private String s;  
    public Contiene(String s) {  
        this.s = s;  
    }  
    public boolean selecciona(Persona p) {  
        return p.getNombre().contains(s);  
    }  
}
```

abstraccionfuncional/Contiene.java

✓ ¡¡¡ No hay que cambiar las otras clases !!!

Principios SOLID:

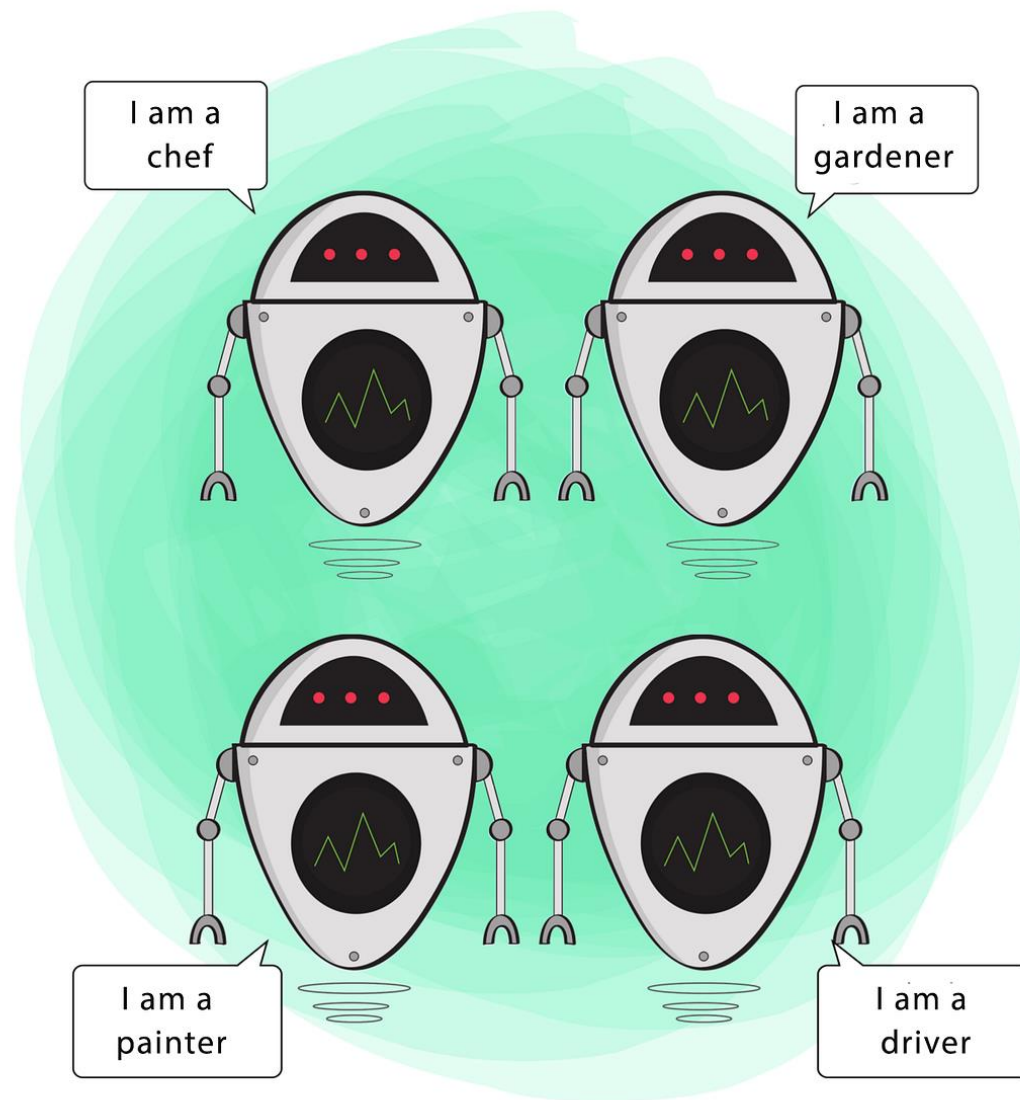
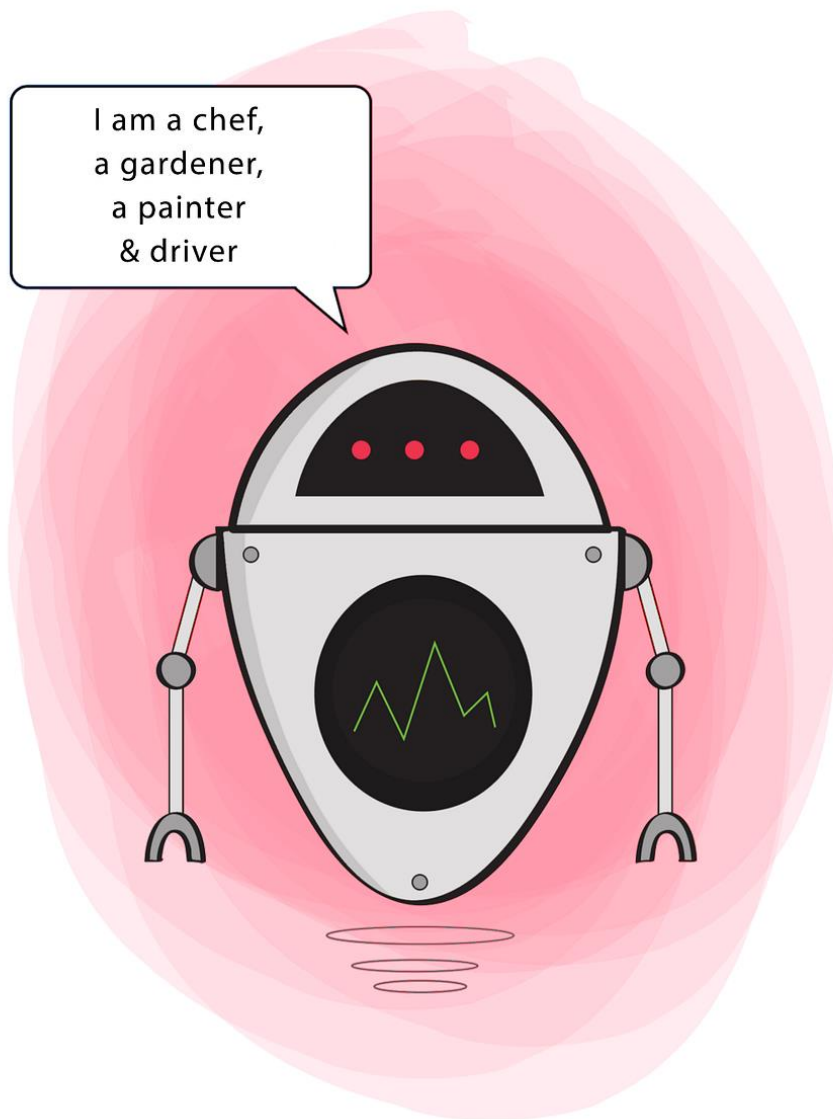
evitemos errores en sistemas en producción



<https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898>

SOLID

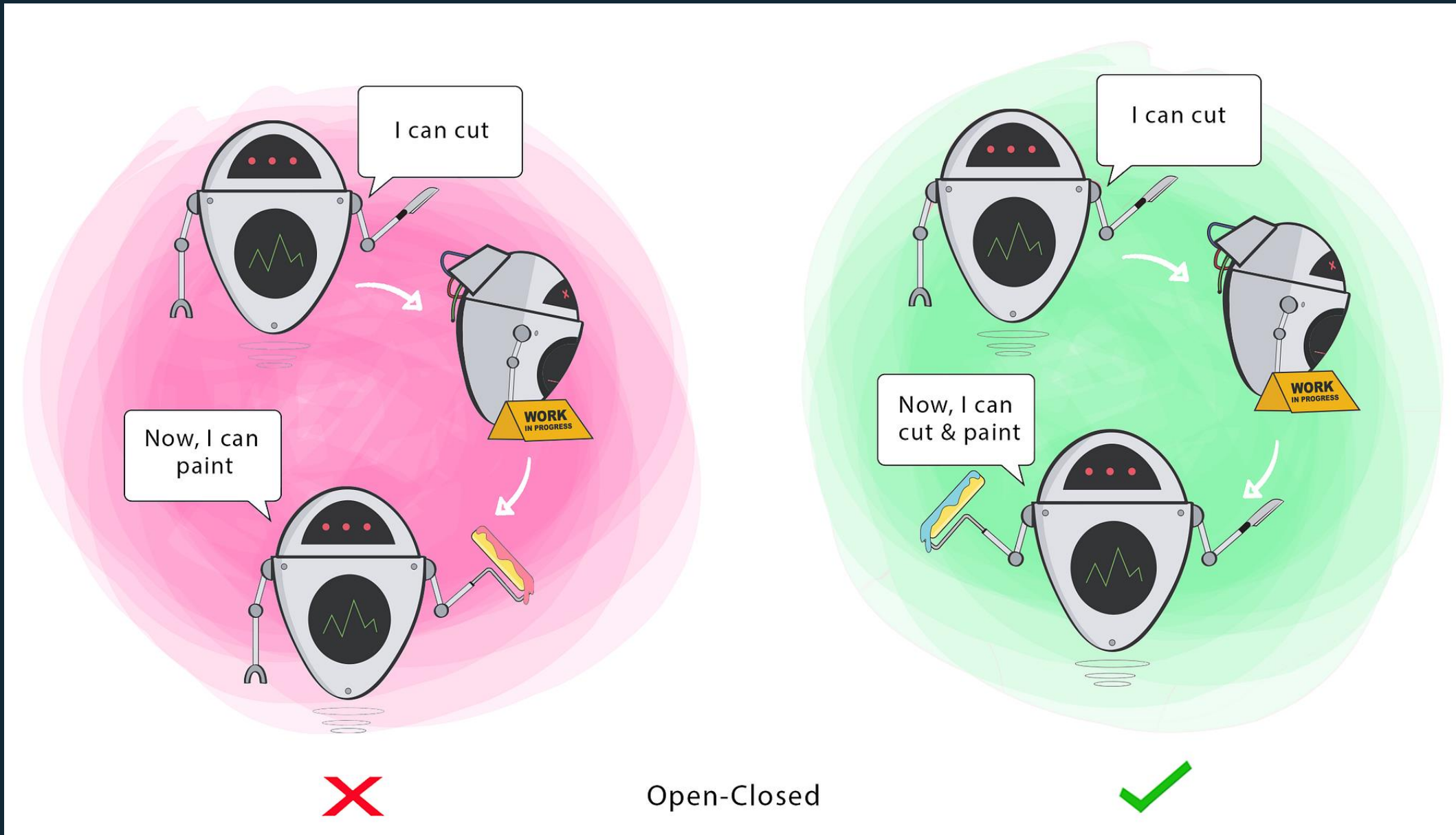
Una clase debe tener una única responsabilidad.



Single Responsibility

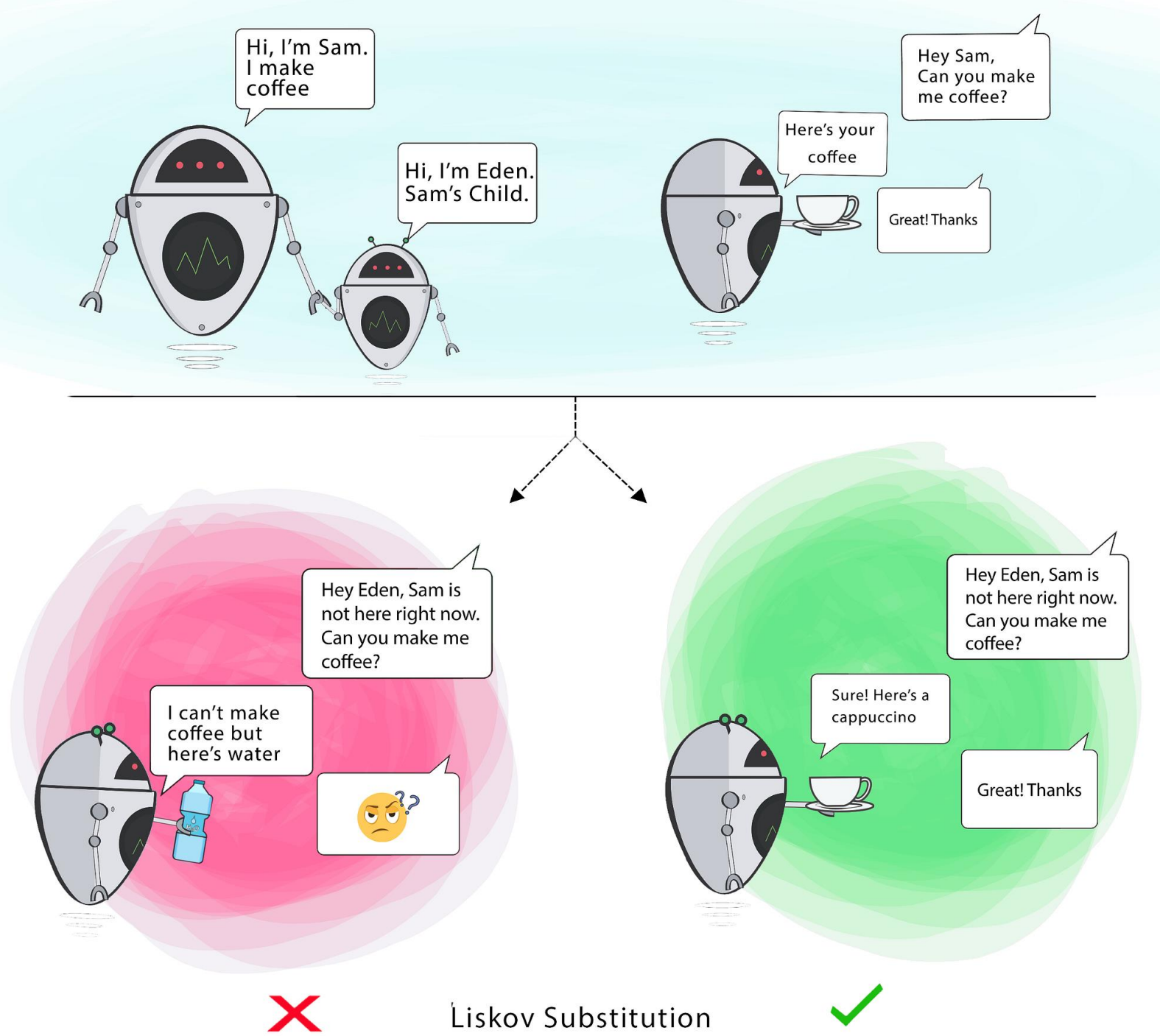
SOLID

Las clases deben estar abiertas para su extensión, pero cerradas para su modificación.



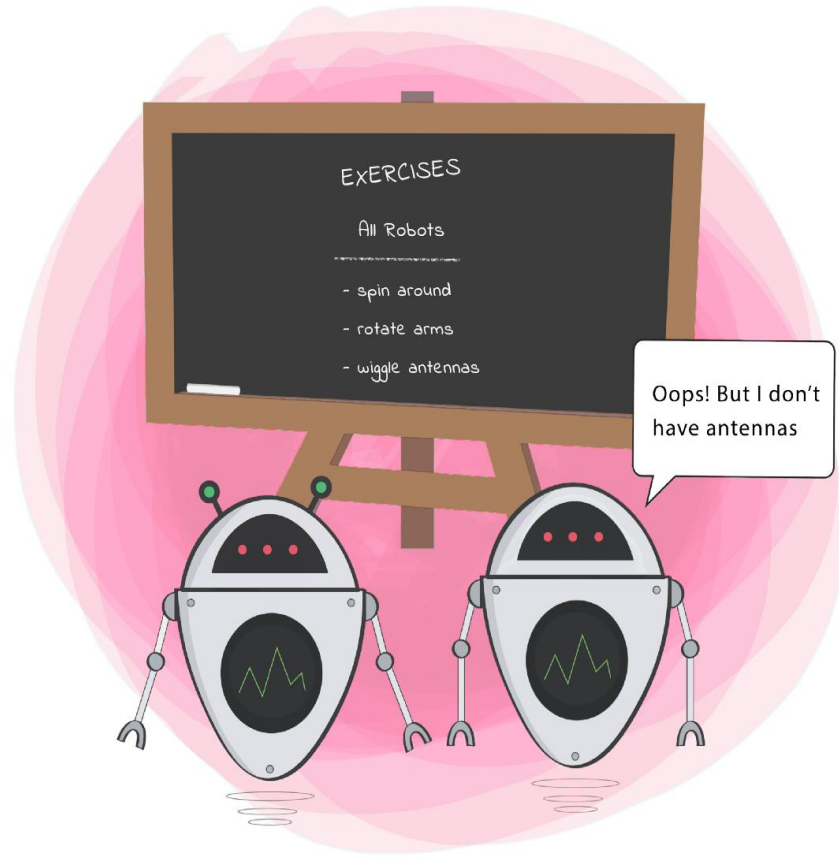
SOLID

Si S es un subtipo de T, entonces los objetos de tipo T en un programa pueden reemplazarse por objetos de tipo S sin alterar ninguna de las propiedades deseables de ese programa.

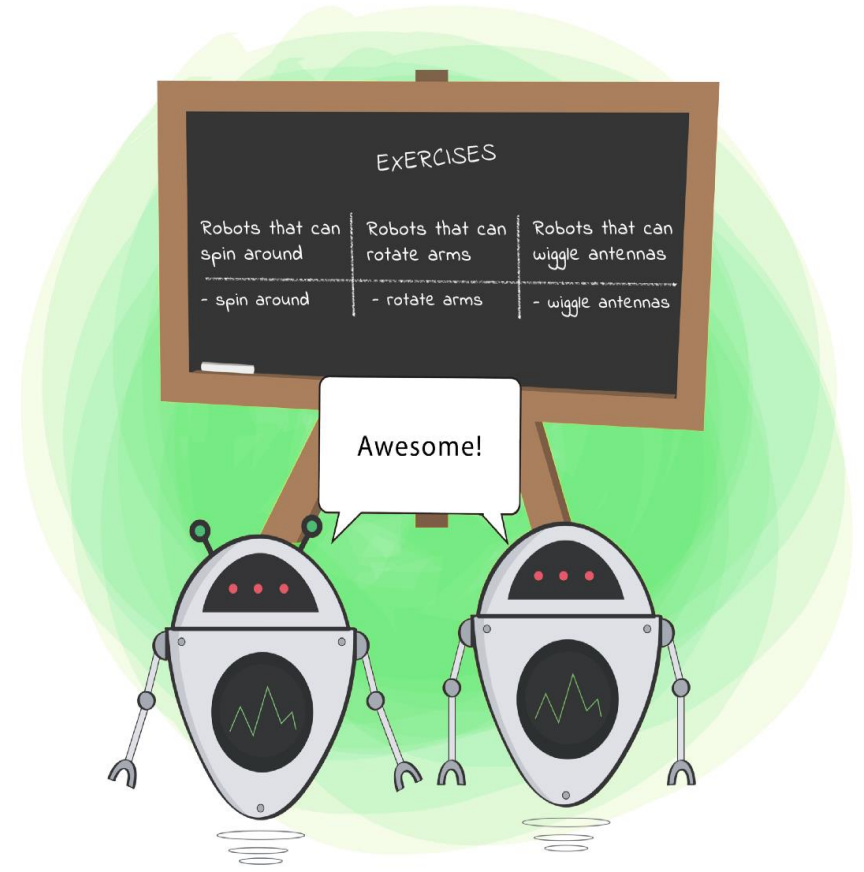


SOLID

Las clases no deberían verse obligadas a implementar métodos que no utilizan.

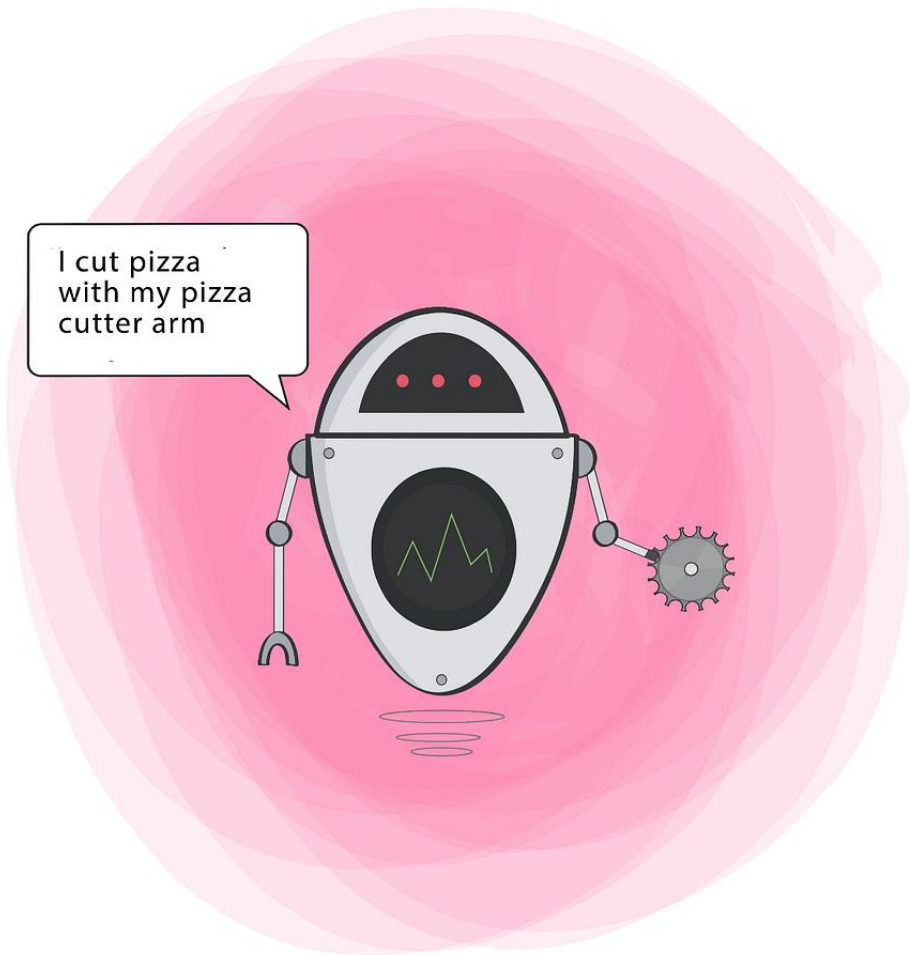


Interface Segregation

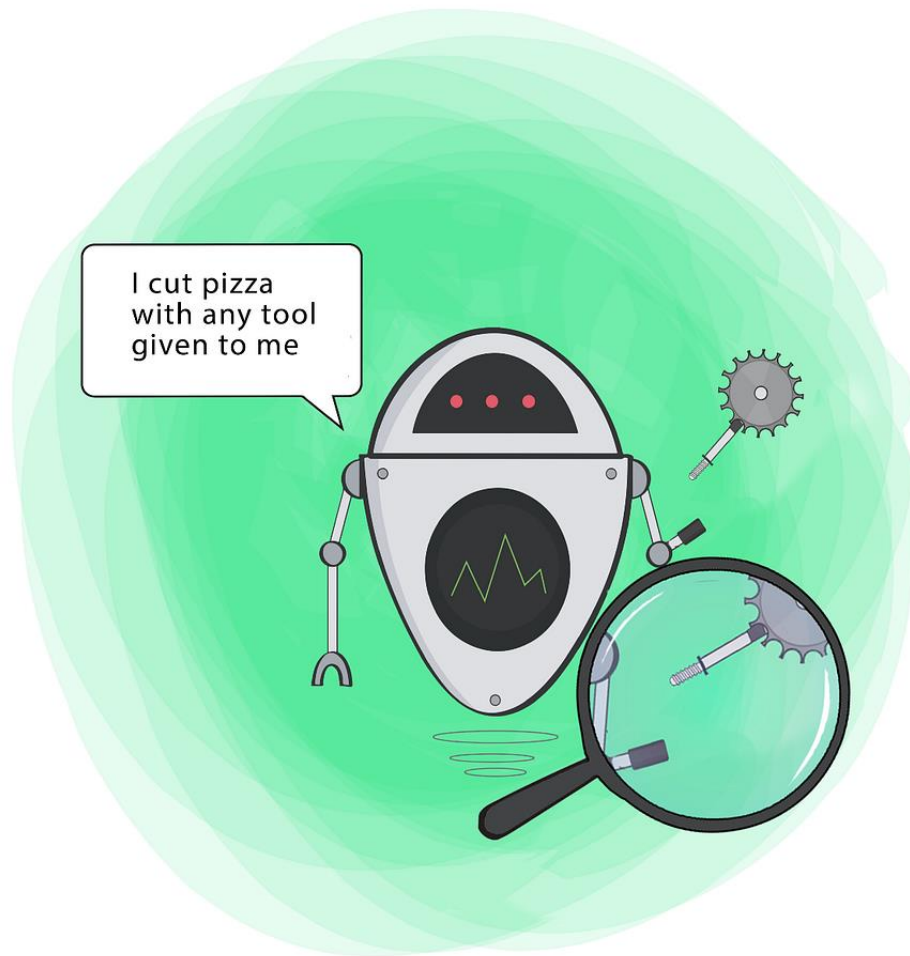


SOLID

Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones.



✗



✓

Dependency Inversion